Retrieval-Augmented Generation

Part 2: combining, reordering, and making it fast

Where we stopped

Last lecture ended with two methods and no winner:

BM25	**Embeddings**
Non-zero score *if and only if* a query word literally appears. On our paraphrase question it scored **exactly 0.000** and landed at rank **17 of 18**.	Never exactly zero, never guaranteed either. On the same question it put the right chunk at rank **4**.

Different **guarantees**, not different strengths. Today: keep both, and then put the survivors in the right order.

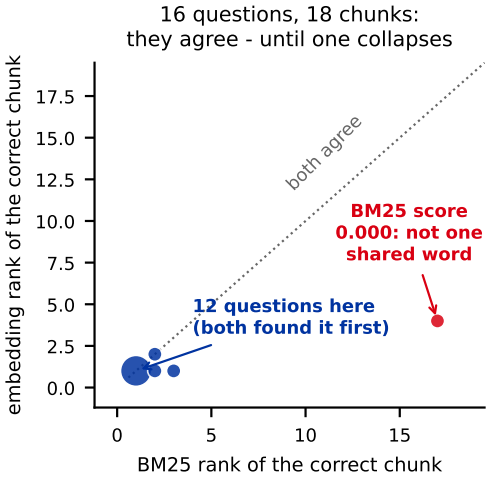
Predict first

We took **16 questions** about the cheese factory,
each with one chunk that answers it,
and ran both methods over the same 18 chunks.

On how many of the 16 do the two disagree?

Commit to a number before the next slide.

The answer: almost never, and then catastrophically



On **15 of 16** questions the two land within two ranks of each other, and on 12 of them both put the answer first. On the sixteenth, BM25 does not degrade - it stops.

The argument for combining is not "each wins different questions". It is that **one of them can fail completely**, and you cannot tell which question in advance.

Three ways to spend the rest of this lecture

1. **Fix the question** before it reaches either method. Real users do not type keywords.
2. **Fuse the two ranked lists** into one, without pretending the two scores are comparable.
3. **Reorder the survivors** with a slower, better model, and make the whole thing fast enough to ship.

Everything measured today runs on an **18-chunk toy corpus** with a strong multilingual embedder. That is the setting where these techniques have the *least* to offer. Watch what the numbers actually say, not what the technique is supposed to do.

Outline

Fixing the question

Fusing two ranked lists

Filtering before you rank

Reranking

Making the search fast

Fixing the question

Nobody types keywords. They type a paragraph with an apology in it.

What your user actually sends

> *"Sorry to bother you again. Last week the hydraulic gauge on line two was reading strangely during the second pressing stage and the operator logged it in the maintenance report. I could not find the number anywhere, so what pressure is the Lori press actually supposed to run at?"*

Forty-nine words. The question is **eleven** of them, at the very end. The rest is a story about last week, and it is full of words that match the *wrong* chunks: `gauge`, `maintenance`, `logged`, `line two`.

Keyword extraction means: pull the searchable terms out of this, and search with those instead.

Three families, and they disagree about what a keyword is

Statistical

Count things. No model, no training, no network call. Microseconds.

`RAKE` and `YAKE`, both spelled out in a moment

Embedding-based

Which phrase is closest in meaning to the whole text? Needs the encoder you already have.

`KeyBERT`

Just ask a model

"List the search terms." Best output, worst latency, and it costs money per question.

We will run all three on that one question, then **measure what each does to retrieval**. The ranking of the three is not what you would guess.

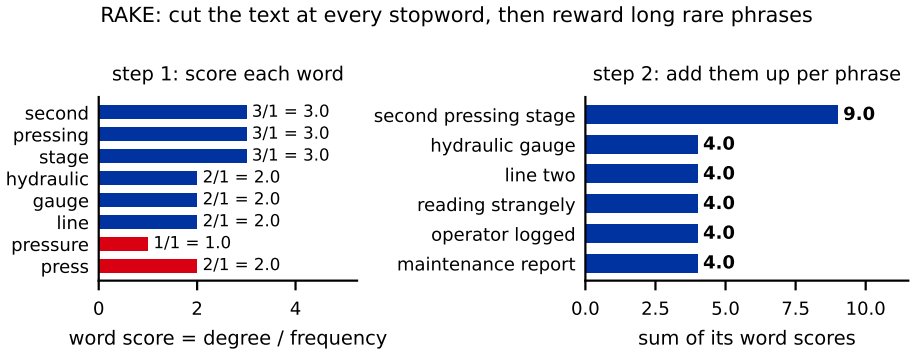
RAKE: cut at the stopwords

Rapid Automatic Keyword Extraction (**RAKE**), from around 2010. Three steps, no training data:

1. Split the text at every stopword and punctuation mark. What survives between the cuts is a candidate phrase.
2. For each word: $\text{score}(w) = \text{degree}(w)/\text{freq}(w)$, where degree counts how many words share a candidate phrase with it, itself included - not just its immediate neighbours.
3. A phrase scores the sum of its words.

Read step 2 again: a word in a **long** phrase gets a high degree. So RAKE systematically prefers long multi-word phrases over single important words.

RAKE on our question



red = the phrase contains "pressure", "press" or "Lori" - the words that point at the answer

"second pressing stage" is three words that each stand next to two others, so each scores $3/1 = 3$ and the phrase scores 9. Meanwhile `pressure` sits alone between two stopwords: $1/1 = 1$, dead last.

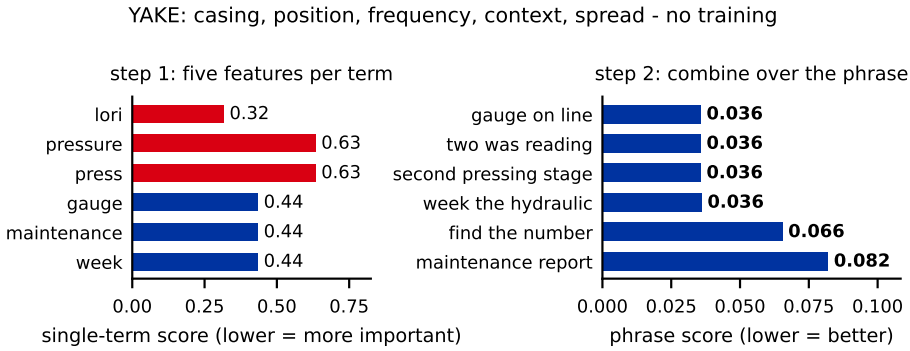
YAKE: five statistics, no training

Yet Another Keyword Extractor (**YAKE**), around 2018. It scores each term by combining five features, all computable from the single document in front of it:

Feature	**Intuition**
casing	words written in capitals or as acronyms tend to matter
position	terms that appear early tend to matter
frequency	normalised against the document's own average
context	a term with *few* distinct neighbours is more specific
spread	appearing in many sentences means it is a topic, not an aside

YAKE reports **lower is better**. Every one of the five features is about the text *itself* - none of them knows what a cheese press is.

YAKE on our question



red = the phrase contains "pressure", "press" or "Lori" - the words that point at the answer

`pressure` and `press` score **0.63**; `gauge`, `maintenance` and `week` score **0.44** and therefore win. They arrive earlier in the text, and position is one of the five features.

YAKE, by hand

Five features per word, then one combination. Real values from our question:

word	case	pos	freq	rel	diff	score
`lori`	**1.000**	0.476	1.000	1.333	0.333	**0.317**
`week`	0.000	**0.327**	1.000	1.333	0.333	0.436
`press`	0.000	0.476	1.000	1.333	0.333	0.635
`pressure`	0.000	0.476	1.000	1.333	0.333	0.635

$$\text{score} = \frac{\text{rel} \times \text{pos}}{\text{case} + \text{freq/rel} + \text{diff/rel}}$$

lower is better

What that table quietly admits

Look down the columns. `freq`, `rel` and `diff` are **identical for every word**: 1.000, 1.333, 0.333.

On a 49-word question, every content word appears once, in one sentence, with one neighbour on each side. Three of the five features have nothing to distinguish.

So on a short query, YAKE is not really using five statistics. It is using **casing and position**, and the rest is along for the ride. `lori` wins here purely because it is capitalised mid-sentence.

YAKE was designed for documents, not for questions. Applying it to a one-paragraph query is using it outside the setting it was built for - which is worth knowing before you reach for it.

KeyBERT: ask the encoder instead of counting

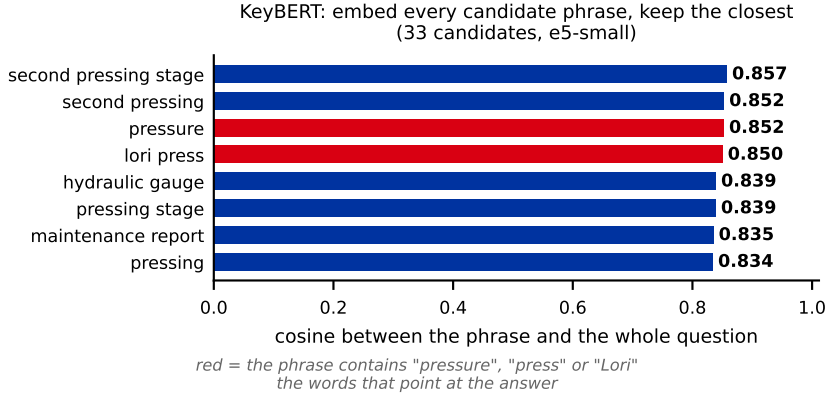
A small library from 2020, and the whole algorithm fits on this slide:

1. Embed the whole question into one vector.
2. Take every candidate phrase of 1 to 3 words. Embed each one.
3. Keep the phrases whose vector is closest to the question's vector.

It reuses the embedding model you already loaded for retrieval, so it costs one extra batch of very short texts. No new model, no training.

Because it works in meaning space, a phrase can win without being frequent, early, or long.

KeyBERT on our question



`pressure` is now third and `lori press` fourth, out of 33 candidates. Neither statistical method put them anywhere near the top.

Or just ask a language model

Prompt: "Extract the search terms from this question. Return only the terms."

```
Lori press   pressure   operating pressure
```

Cleanest output of the four. It threw the story away, which is exactly what you wanted. What it costs you:

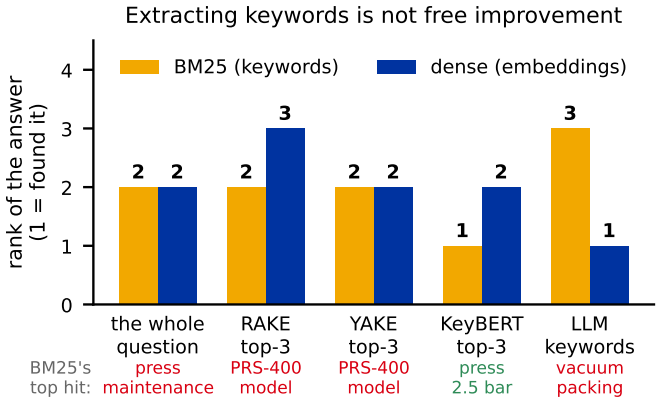
- A network round trip **before** the search even starts.
- Money on every question, forever.
- Non-determinism: the same question can produce different terms tomorrow.
- It can invent a term that appears *nowhere* in your documents.

Four keyword sets, one target chunk, the same 18 documents.

**Which of them finds the answer,
and does any of them beat sending
the whole rambling question?**

Vote for one.

Measured, and it is not a clean win



The raw question already reached rank 2. **RAKE and YAKE bought nothing**: the answer stayed at rank 2, and both handed BM25 the phrase "line two", which pushed the chunk about *model PRS-400 on line two* into first place. So the answer did not move, but what sits above it got worse. Only KeyBERT reached rank 1.

Why the cleanest keywords lost

The language model returned `Lori press pressure operating pressure`. BM25 ranked the correct chunk **third**, behind the vacuum packer.

> The answer chunk says "…operates at 2.5 bar during the first **pressing** stage." It never contains the word **pressure**. The vacuum-packer chunk does.

Worse: the model said `pressure` twice, and our scorer sums over query terms, so that word counted double.

The vocabulary-mismatch problem from last lecture reappears **inside** the keyword extractor. A perfect human-sounding keyword is still just a string, and BM25 only matches strings.

What to take from that experiment

- Keyword extraction is a **rewrite of the query**. Any rewrite can help or hurt, and which one it does is an empirical question about *your* corpus.
- The statistical methods optimise for "what is this text about". A rambling question is mostly about last week.
- The meaning-based method did best here, because it is the only one that knows `pressure` and `press` are related.
- Extraction matters most for the **keyword half** of a hybrid system. The embedding half was barely affected: it handled the raw question fine.

Ship it behind a measurement, never on the argument that it sounds sensible.

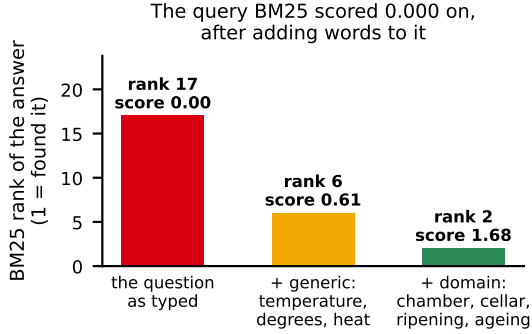
The opposite move: add words instead of removing them

Query expansion. Keep the question, and append terms it *should* have used.

Source of the extra terms	**Example**
a hand-written synonym list	`press` → `press, mould, platen`
the other language your users type in	the Armenian for `pressure` → `pressure, bar`
a language model, per query	"give me five ways to say this"
the top result of a first search	take its words, search again

In a bilingual factory this is not optional. The manuals are in English and the engineer asks in Armenian, so *every* content word misses and BM25 scores **exactly zero** until somebody bridges the two vocabularies.

Expansion on the question BM25 could not see



Generic synonyms (`temperature`, `degrees`, `heat`) lift it from rank 17 to 6; domain words (`cellar`, `ripening`, `ageing`) reach rank 2.

The added terms are synonyms of *the question's own words*. Picking words out of the answer would prove nothing.

Expansion has a cost too

Every word you add is a word that can match something irrelevant.

What you buy

Recall. A question phrased in the wrong vocabulary stops scoring zero.

What you pay

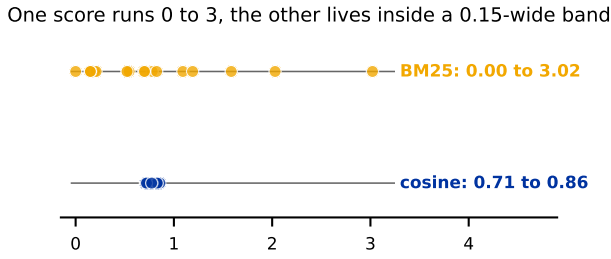
Precision. `heat` also matches pasteurisation, and `ageing` matches the maintenance schedule.

Expansion is a **recall** tool. It is safe precisely when something later in the pipeline can throw the extra rubbish away again - which is what the last two sections of this lecture are for.

Fusing two ranked lists

Two searches, two orderings, one answer box. The obvious way to combine them is wrong.

The obvious idea: add the scores



BM25 has **no upper bound**: it grows with the number of query terms and their rarity. Cosine is stuck in $[-1, 1]$, and for a real encoder almost everything lands in a narrow band near the top. Add them and the bigger number decides everything.

Then normalise them first?

The standard fix is min-max: rescale each list to $[0, 1]$ before adding. It breaks in three ways, all of them quiet:

1. **The scale depends on the batch.** The best score in *this* result set becomes 1.0, whether it was a brilliant match or the least bad of eighteen failures.
2. **One outlier rescales everybody.** A single very high BM25 score squashes the rest of that list toward zero.
3. **It destroys the one honest signal BM25 has.** A score of exactly zero means "no query word appears here". After normalisation it is just the bottom of the range.

Each of those is easy to say and easy to disbelieve. So here they are, measured.

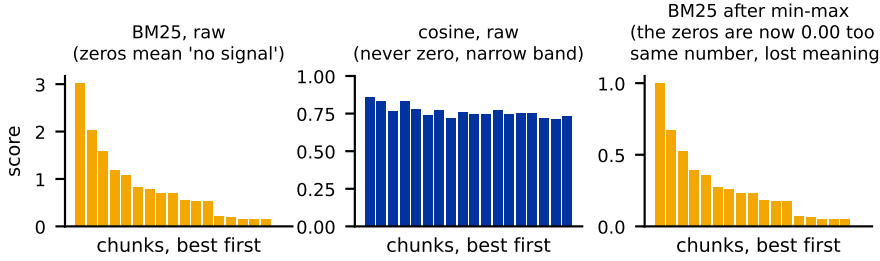
The same three failures, with numbers

One query, our eighteen chunks. BM25 runs from **0.000** to **3.021**; cosine from **0.710** to **0.858**.

1. Batch-dependent	One chunk, unchanged, scored twice: **0.394** among all 18 candidates, **0.211** among the top 8. Same chunk, same query, different number.
2. Outliers squash	Triple only the top score and second place collapses from **0.672** to **0.224**, without moving.
3. The honest zero dies	Min-max sends the *worst* chunk to 0.00 - whether it scored 0.000 (no query word appears at all) or 1.5 (a real, weaker match).

The third is the one that should bother you most. **BM25's zero means "not in the language of this chunk". Min-max's zero means "last in this batch".** Those are different facts, and normalisation prints them as the same number.

What normalisation does to the picture



So throw the scores away and keep only what is comparable between the two systems: the **order**.

Reciprocal Rank Fusion

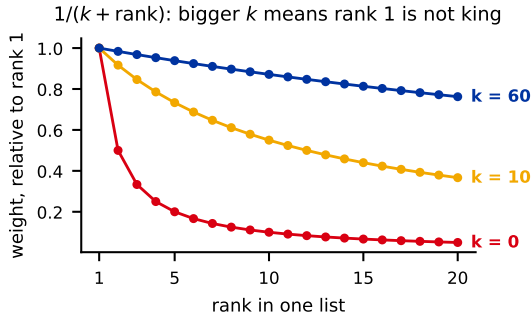
Reciprocal Rank Fusion (**RRF**), from a 2009 information-retrieval paper by Cormack and colleagues. One line:

$$\mathrm{RRF}(d) \;=\; \sum_{\text{lists } i} \frac{1}{k + \mathrm{rank}_i(d)}$$

A document collects a small contribution from every list it appears in. Nothing else is used: not the score, not the units, not the range.

- It extends to **any** number of lists, including a third one from a different index.
- A document missing from a list simply contributes nothing from it.
- k is a constant, conventionally **60**.

What the constant k is for



With $k = 0$, rank 1 is worth twice rank 2 and one list can dictate the result. With $k = 60$ the weights are nearly flat: rank 1 gives 0.0164 and rank 2 gives 0.0161.

Large k says: *being in both lists matters more than being first in one of them.* That is the entire design.

By hand: two lists for one question

"Are incoming deliveries checked for drug residues?" the answer is the milk-testing chunk

Rank	BM25	Embeddings
1	PRS-410 spare parts	**milk testing**
2	ripening cellar	press maintenance
3	**milk testing**	gloves and knives
4	PRS-380 press	vacuum packing
5	pasteurisation	pasteurisation

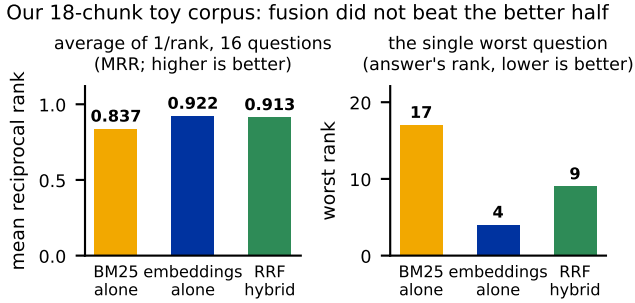
Five of the seven question words appear in *no* chunk at all. BM25's top two matched on `for` and `are` and nothing else.

By hand: the fusion, $k = 60$

Chunk	BM25	Dense	Fused score
milk testing	3	1	$\frac{1}{63} + \frac{1}{61} = 0.0159 + 0.0164 = \mathbf{0.0323}$
pasteurisation	5	5	$\frac{1}{65} + \frac{1}{65} = 0.0154 + 0.0154 = \mathbf{0.0308}$
PRS-410 spare parts	1	-	$\frac{1}{61} + 0 = \mathbf{0.0164}$
ripening cellar	2	-	$\frac{1}{62} + 0 = \mathbf{0.0161}$
press maintenance	-	2	$0 + \frac{1}{62} = \mathbf{0.0161}$

Fifth in both beats first in one. Pasteurisation, mediocre twice, ends up above the chunk BM25 was most confident about. And the right answer moves from rank 3 to rank 1 without anyone tuning a weight.

Now run it on all 16 questions



Fusion beat BM25 comfortably and **did not beat the embeddings alone**: 0.913 against 0.922. On the one question where BM25 was blind, fusion took the answer from rank 4 to rank **9**.

Those two numbers are the same fact. That single question costs $(\frac{1}{4} - \frac{1}{9})/16 \approx 0.009$ of the average - which is the entire gap. So fusion is not quietly worse everywhere; it is **fine on 15 questions and bad on one**.

Why fusion lost here, and what that teaches

RRF sees ranks. It cannot see that BM25's ranking on the paraphrase question was **pure tie-breaking noise**, because every score there was 0.000.

> A rank-based method treats a confident list and a meaningless list identically. That is the price of being scale-free.

Practical repairs, in the order you should try them:

- Fuse only each list's **top-k**, so the noisy tail never votes.
- Drop a list entirely when its top score is zero or near-zero.
- Weight the lists, once you have a measurement that justifies the weights.

So when is hybrid worth it?

Eighteen chunks and an embedder that is good at this domain is the worst possible advertisement for hybrid search. It earns its place when:

- the corpus is large enough that a near-miss neighbour is always there to outrank the right chunk;
- queries contain **part numbers, clause references, surnames** - strings with no meaning to embed;
- the vocabulary is outside the embedder's training data: internal jargon, product names, abbreviations your factory invented;
- **your language is not well covered** - the Armenian case from last lecture. A weak embedder makes the lexical half essential.

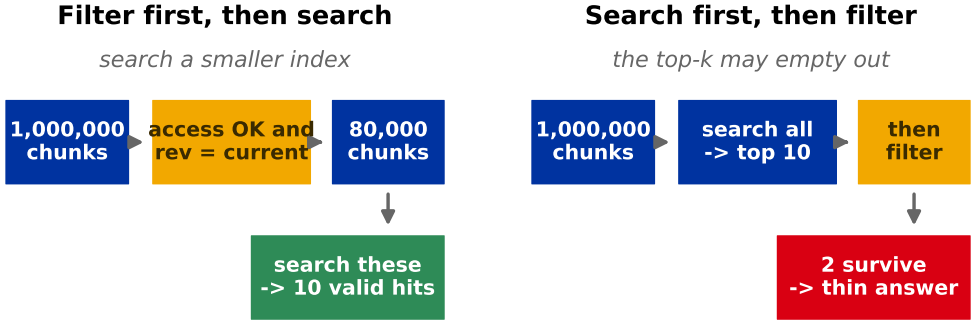
Add the second retriever because you can name the queries it rescues, then check that it did. "Everyone uses hybrid" is not a measurement.

Filtering before you rank

The fastest way to improve a search is to stop searching things that were never eligible.

Metadata filtering: the cheapest quality win there is

Where the filter goes changes what the user gets



Illustrative counts - the ordering is the point, not the numbers.

The fields we stored back in the chunking section - access level, revision date, document type - finally do something.

Filtering is where access control actually happens

In Section 1 of the last lecture we said a model has no permissions. Here is the mechanism:

Filter	What it prevents
`access_level` $\leq$ `user's level`	the intern retrieving the salary table
`revision = current`	answering from a 2019 procedure
`doc_type` $\neq$ `draft`	citing something nobody approved
`date` $>$ 2023	quoting a price list from before inflation

Do the permission filter **before** the search, not after. Filtering afterwards means the forbidden chunk was retrieved, scored, and possibly logged - and your ten results have quietly become two.

Reranking

The fast model gets you a shortlist. Something slower and smarter decides the order.

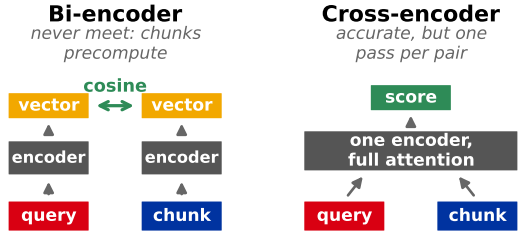
The bottleneck built into the bi-encoder

A bi-encoder compresses a chunk into one vector **before it has ever seen your question**. That vector has to serve every question anyone will ever ask.

> "Model PRS-400 replaces the older PRS-220 press on line two."
>
> One vector, and it must answer *"what replaced the PRS-220?"*, *"what is on line two?"* and *"which presses are new?"* equally well.

The query and the chunk **never look at each other**. Speed comes from exactly that, and so does the ceiling on quality.

The fix, and why you cannot afford it



The **cross-encoder** reads question and chunk together, with attention running across both. It is the same transformer, used differently.

There is no vector to precompute, so nothing can be done offline. Every (question, chunk) pair needs its own forward pass **at query time**.

Put a number on “cannot afford”

Measured on this laptop’s CPU, with a small 12-layer encoder standing in for a cross-encoder of the same size:

Work	**Time**
encode one question (bi-encoder)	about 40 ms
compare it against 1,000,000 stored vectors	80 to 100 ms
score 100 pairs with the cross-encoder	about 3 seconds
score 1,000,000 pairs with the cross-encoder	**about 9 hours**

Scan measured directly: **82 ms**. Extrapolated from smaller indexes: **100 ms**. A 20 percent spread is normal, which is why every row says “about”.

A better GPU moves every row of this table and does not change its shape. Per document the cheap model spends about **0.0001 ms**, the expensive one about **30 ms**.

The cascade

Retrieve wide with the cheap model, re-score the survivors with the expensive one



Cross-encoding all 1,000,000 chunks instead: 9 hours per query.

Measured on this laptop CPU with e5-small; the 1M scan is the 100,000-vector timing scaled linearly.

Cheap and wide first, expensive and narrow last. This shape is everywhere in production search, and it is the one diagram from this chapter you will reuse.

Two stages is the common case. **Late interaction** (three slides on) is the optional middle one: too slow for a million chunks, fast enough for a few thousand.

Choosing the width of each stage

- **The reranker can only reorder what stage one returned.** If the answer is not in your 100 candidates, no reranker in the world will find it. This is the recall ceiling from the chunking section, one stage later.
- **So widen stage one until recall stops improving**, then stop. Going from 100 to 500 candidates costs you five times the reranking latency.
- **Rerank only what you can pay for.** The cost is linear in the number of candidates and you feel every one of them.
- **The final cut is a token budget**, not a quality judgement: three good chunks in the prompt beat ten mediocre ones.

Measure recall at stage one and precision at stage two. They are different jobs and they fail differently.

A middle option: let the tokens meet, but late

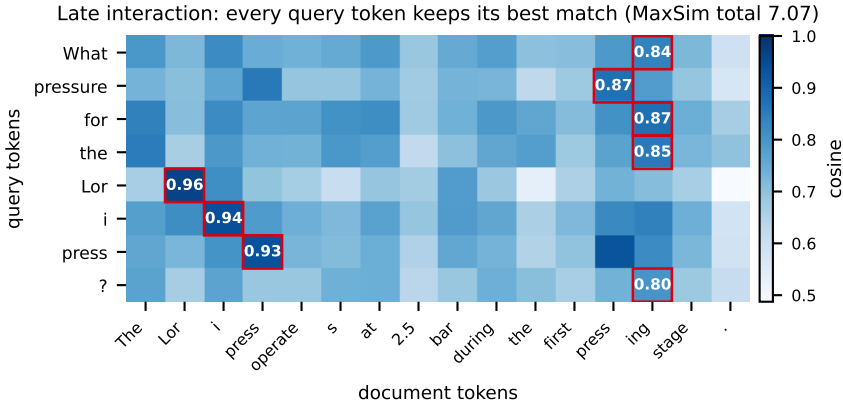
ColBERT (2020) keeps one vector *per token* instead of one per chunk, and compares them only at scoring time. That is called **late interaction**.

$$\text{score}(q, d) \;=\; \sum_{t \in q} \max_{u \in d} \cos(v_t, v_u)$$

Every query token finds its own best partner in the document and keeps that similarity. The name for the inner operation is **MaxSim**, short for maximum similarity.

The document tokens are still precomputed offline. Only the cheap comparison happens per query - so this sits between a bi-encoder and a cross-encoder in both cost and quality.

MaxSim, computed on real token vectors



Lor finds Lor at 0.96 and press finds press at 0.93. Note that pressure matches press at 0.87 - the thing plain keyword matching could never do.

Token vectors from our ordinary encoder, not a trained ColBERT: this shows the mechanism. A model trained for late interaction pushes the unrelated cells much lower than they are here.

Three shapes, one trade-off

	Bi-encoder	**Late interaction**	**Cross-encoder**
query meets chunk	never	at scoring time	inside the model
stored per chunk	1 vector	1 vector per token	nothing
cost per query	one scan	many small products	one pass *per chunk*
used for	search millions	rerank thousands	rerank tens

The single axis is **how late the question meets the document**. Later means better and slower, every time. Pick a different point on that axis at each stage of the cascade.

The case reranking cannot solve

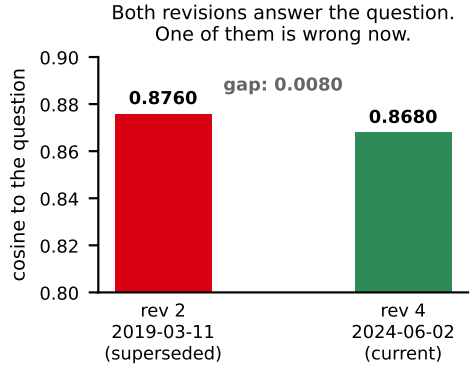
Two chunks from the same manual, both retrieved, both about the Lori press:

> **rev 2**, 2019-03-11: "the Lori press is held at **2.0 bar** for the whole pressing cycle."
>
> **rev 4**, 2024-06-02: "the Lori press starts at **2.5 bar** and is raised to 3.2 bar after twenty minutes."

The question: *"What pressure should the Lori press run at?"*

Both are relevant. Only one is true today.

Measured on the two revisions



The **superseded** revision scores higher, by 0.008. No threshold separates those two numbers, and a cross-encoder would not help: it is asked *is this chunk relevant*, and both chunks honestly are.

What actually fixes it

Relevance is the wrong question. The right one is *which of these is still in force*, and that is a fact about the document, not about its text.

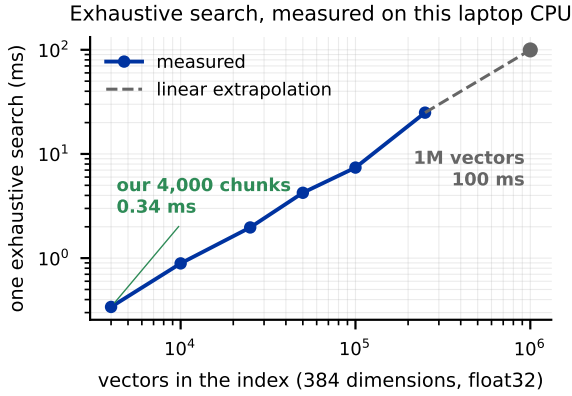
1. **Filter.** Index only the current revision, or filter on it at query time. Cheapest, and correct whenever "current" is well defined.
2. **Sort by metadata after reranking.** Among chunks of comparable relevance, prefer the newest revision.
3. **Pass both to the model, with their dates attached**, and let the answer say "the current procedure (rev 4) is 2.5 bar, raised to 3.2".

This is why the metadata fields went in during chunking. A retrieval system with no revision dates cannot be made correct by a better model, only by a better index.

Making the search fast

Three techniques, in the order you should actually reach for them.

Start by not building an index at all



Comparing a question against *every* vector is one matrix-vector product. Our 4,000-chunk factory archive: well under a millisecond. A million chunks: about a tenth of a second, on a laptop, with no index at all.

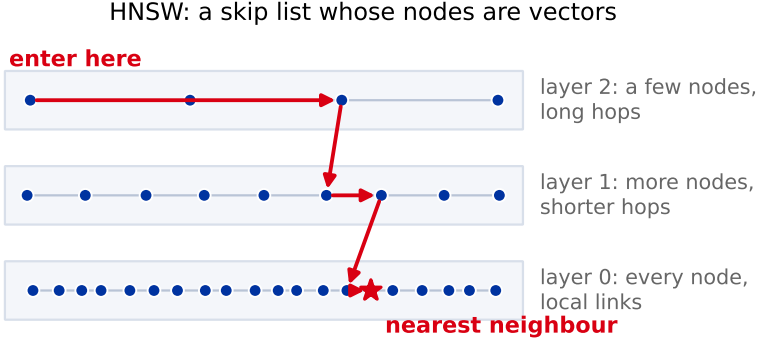
So when does brute force stop being enough?

Usually not because of time. Because of one of these:

- **Memory.** 10 million chunks at 384 dimensions in float32 is about **15 GB**, and it has to be resident to be scanned.
- **Concurrency.** 100 milliseconds per query is fine for one user and hopeless for a hundred at once on the same machine.
- **It is not the only thing you do.** The scan competes with the encoder, the reranker and the model generating the answer.

Under roughly 100,000 chunks, an exact scan is usually the right engineering answer, and it has a property no approximate index has: it **cannot** miss.

HNSW: a skip list in high dimensions



Schematic - a real graph has 384-dimensional nodes and no left-to-right order.

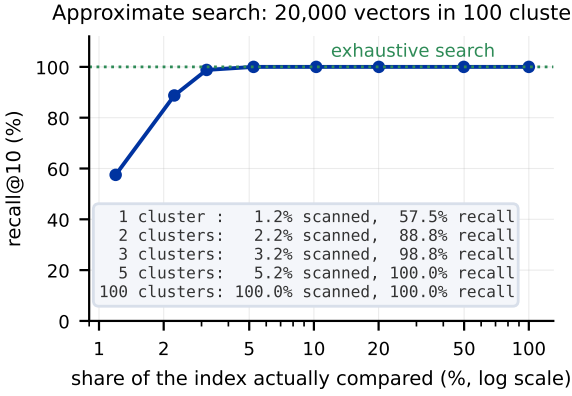
Hierarchical Navigable Small World (**HNSW**), 2016. Each vector is a node linked to its near neighbours. A few nodes are copied into sparse upper layers, so a search starts with long jumps and finishes with short ones.

Why that shape is fast

- Search enters at the top layer, walks greedily to the closest node it can see, then drops a layer and repeats. Each layer refines the previous one's answer.
- Cost is roughly **logarithmic** in the number of vectors instead of linear. Ten times more documents is a few more hops, not ten times the work.
- The same idea as a skip list over a sorted array - except there is no sorted order in 384 dimensions, so the graph's edges have to stand in for one.

Building the graph is not free: it is memory-hungry and insertion is much slower than appending to an array. Deleting is worse - most implementations only mark nodes as removed and need a periodic rebuild.

Approximate means approximate



Approximate Nearest Neighbour (**ANN**) search, measured here in its simplest form, an **inverted file** (**IVF**): group the vectors, then look inside only the nearest few groups.

Scanning **3.2%** of this index recovered **98.8%** of the true top 10. Scanning 1.2% recovered 57.5%.

Every ANN index has a knob that trades recall for speed. The default is somebody else's compromise, and it never announces the documents it silently skipped.

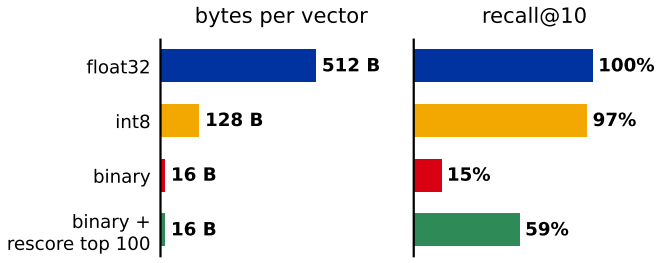
Know your recall number

It costs one afternoon and it is the only way to know what your index is doing:

1. Take a few hundred real queries.
2. Run them through your approximate index, keeping the top 10.
3. Run the same queries through an **exact** scan. On a sample of the corpus if the whole thing does not fit.
4. Report the overlap. That fraction is your recall.

Then turn the knob until recall is where you want it and latency is what you can pay. A system at 85% recall is not broken - but you should be the one who chose 85%.

Quantization: the same vectors, smaller



128-dimensional vectors. Store each dimension in one byte instead of four: a quarter of the memory, and 97% of the true top 10 survived. Store one *bit* per dimension and the ranking mostly collapses - rescoring the shortlist in full precision recovered part of it.

Measured on synthetic clustered vectors. Real embedding models tolerate binary quantization better than this; the discipline of measuring what it cost you is the transferable part.

The order to reach for these

Corpus size	What to use	What to check
under 100k chunks	exact scan	nothing, it cannot miss
100k to 10M	HNSW, float32	recall against an exact sample
beyond that	HNSW + int8, rescore the top	recall, and memory per node

Every row down this table trades a guarantee for a resource. Take the trade deliberately, with the recall number in your hand, and not one row earlier than you have to.

Recap

- **Fix the question first.** Extraction is a rewrite: it helped once in four tests and hurt twice. Expansion took a 0.000 score from rank 17 to rank 2.
- **Never add BM25 and cosine.** Unbounded plus narrow: the sum is just BM25.
- **RRF fuses ranks, not scores.** Fifth in both beats first in one. Here it beat BM25 and not the embeddings.
- **Filter on metadata**, before the search rather than after.
- **Cascade:** millions of chunks, hundreds of candidates, tens reranked, three in the prompt. Late interaction is better and slower.
- **Reranking cannot fix everything.** Both revisions are relevant; the date decides.
- **Brute force is fine to about 100k chunks.** Past that, know your recall number.

Next: the retrieval is done. Now the model has to use it - assembling the context, forcing citations, and measuring whether any of this worked.